

1 Отчет по задаче 1: Извлечение и валидация грамматики РБНФ

В рамках первого пункта задания я выделил формальную грамматику DSL и оформил её в формате РБНФ. Полный текст сохранен в файле `dsl_grammar.rbnf`.

Для валидации грамматики (чтобы убедиться, что она реально соответствует текущей реализации) я разобрал исходники лексера (`lexer.py`) и синтаксического анализатора (`recursive_descent_parser.py`).

Ниже привел описание того, в каких именно методах кода распознаются ключевые куски грамматики.

Правило грамматики	Метод в коде	Особенности реализации
<Запрос>	<code>parse_query()</code>	Работает как switch по первому токenu: жестко ветвит выполнение на одну из трех функций ниже.
<Запрос CONTEXT>	<code>parse_context_query()</code>	Строгий порядок парсинга: сначала парсится блок <code>WITHIN</code> , а только потом <code>WHERE</code> .
<Запрос FIND>	<code>parse_find_query()</code>	В отличие от <code>CONTEXT</code> , порядок обратный: сначала вызов <code>parse_where_clause()</code> , а уже затем <code>parse_within_clauses()</code> .
<Запрос DISTANCE>	<code>parse_distance_query()</code>	Ожидает два селектора (через токен <code>T0</code>) и специфичный блок <code>LIMIT_PAIRS</code> .
<Условие WHERE>	<code>parse_where_clause()</code>	Делегирует разбор логики в <code>parse_boolean_expression()</code> .
<Блок LIMIT_PAIRS>	<code>parse_limit_pairs_clause()</code>	В коде жестко зашиты проверки на строковые константы <code>"nearest"</code> , <code>"all_nearest"</code> , <code>"all"</code> .
<Блок RETURN>	<code>parse_return_clause()</code>	Собирает элементы вывода. Под функцию <code>distance()</code> в коде написана отдельная проверка круглых скобок.
<Дизъюнкция>	<code>parse_disjunction()</code>	Задаёт низший приоритет для <code>OR</code> .
<Конъюнкция>	<code>parse_conjunction()</code>	Задаёт приоритет для <code>AND</code> .